PROGRAMMING & CS FUNDAMENTALS

# Graphs
# Visual Guide

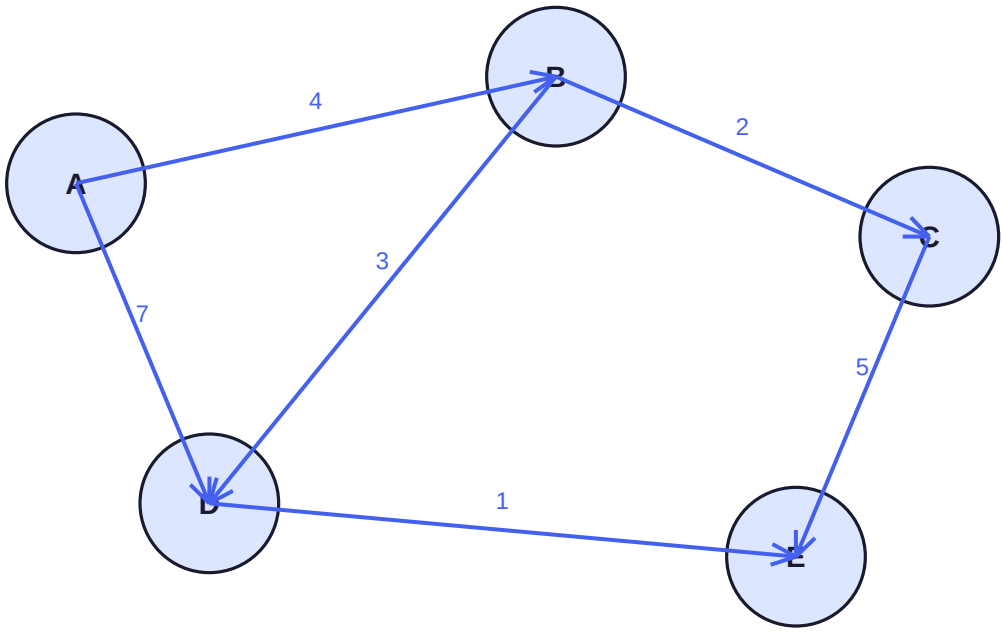
Nodes and edges —
the model behind maps, networks, and dependencies.

**ANIMATED WALKTHROUGH**
Step through pages to play the animation

STEP 1/7

# Anatomy of a Graph

Vertices connected by edges — directed or not, weighted or not.




- Directed: edges have arrows (web links, dependencies). Undirected: mutual (friendships, roads).
- Weighted: edges carry costs (distance, latency, price) — unweighted = every edge costs 1.
- Degree, path, cycle, connected component — the basic vocabulary.
- Dense vs sparse decides the representation (next step).

STEP 2/7

# Adjacency List vs Matrix

The representation IS the performance.

### Adjacency LIST — sparse graphs (E << V^2)

```
A -> [B, D]      # only real edges stored
B -> [A, C, D]
C -> [B, E]
space O(V+E); neighbors of u in O(degree(u))
```

### Adjacency MATRIX — dense graphs

```
    A  B  C  D  E
A [ 0  1  0  1  0 ]
B [ 1  0  1  1  0 ]   # edge? O(1) lookup
space O(V^2) — wasteful when sparse
```

- Default to adjacency lists — real graphs (social, web, road) are sparse.
- Matrix wins for dense graphs and O(1) edge-existence checks.
- Edge list (u,v,w triples): simplest for Kruskal's and streaming.

STEP 3/7

# BFS: Breadth-First Search

Wave outward level by level — shortest path in unweighted graphs.

From A: visit order A -> B -> D -> C -> E (level by level)

```
queue = deque([start]); visited = {start}
while queue:
    u = queue.popleft()          # FIFO = level order
    for v in adj[u]:
        if v not in visited: visited.add(v); queue.append(v)
```

**queue**

FIFO

**visited set**

no repeats

**O(V + E)**

each vertex + edge touched once

First time BFS reaches a node = fewest edges from start. That single property powers shortest-path, 6-degrees, and bipartite checks.

STEP 4/7

# DFS: Depth-First Search

Plunge down one path before backtracking — recursion's natural walk.

```
def dfs(u):
    visited.add(u)
    for v in adj[u]:
        if v not in visited: dfs(v)   # go deep first
# iterative version: swap the queue for a STACK
```

From A (adj order B,D):  A -> B -> C -> E -> D   (dives before spreading)

- BFS vs DFS: same O(V+E), different order — queue spreads, stack dives.
- DFS powers: cycle detection, topological sort, connected components, maze solving.
- Recursion depth = longest path — deep graphs overflow the call stack; go iterative.
- Colors (white/gray/black) track discovery state for cycle detection in directed graphs.

STEP 5/7

# Dijkstra: Shortest Weighted Path

Greedy + priority queue: always settle the closest unsettled node.

```
dist = {all: INF}; dist[start] = 0
pq = [(0, start)]                       # min-heap by distance
while pq:
    d, u = heappop(pq)                  # closest unsettled
    if d > dist[u]: continue
    for v, w in adj[u]:                 # relax edges
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w; heappush(pq, (dist[v], v))
```

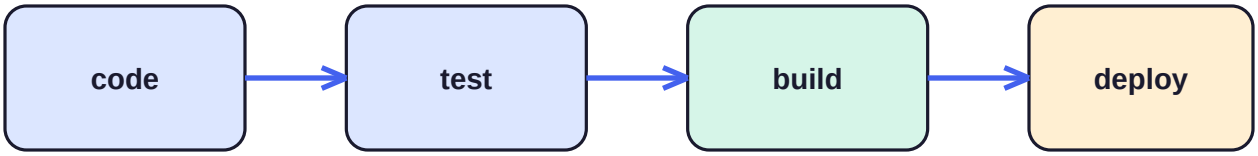
A->E: A-B(4)-C(2)-E(5) = 11  vs  A-D(7)-E(1) = 8  ->  Dijkstra settles D first, finds 8.

- O((V+E) log V) with a binary heap — the log is the priority queue.
- REQUIRES non-negative weights: negative edges need Bellman-Ford (O(VE)).
- GPS routing, network latency, flight pricing — all Dijkstra underneath.

STEP 6/7

# Topological Sort: Order the Dependencies

Linearize a DAG so every edge points forward — builds, courses, task plans.




```
Kahn's: repeatedly emit nodes with in-degree 0, delete their edges
DFS-based: postorder of a DFS, reversed
leftover nodes with in-degree > 0  =>  CYCLE (no valid order!)
```

- Only DAGs (directed acyclic) have a topological order — cycles make it impossible.
- Make, course prerequisites, spreadsheet calc chains, migration ordering.
- Cycle detection via topo sort doubles as a 'is my dependency graph sane?' check.

STEP 7/7

# Graph Algorithm Chooser

Match the problem to the walk.

**BFS**

Shortest path, unweighted — O(V+E)

**Dijkstra**

Shortest path, weighted + — O((V+E) log V)

**Bellman-Ford**

Shortest path, negative w — O(VE)

**Floyd-Warshall**

All pairs — O(V^3)

**Topo sort (Kahn/DFS)**

Dependency order — O(V+E)

**Kruskal / Prim (MST)**

Min cost to connect all — O(E log E)

STEP ✓I✓

# Key Takeaways

## Graphs — Remember

- Model: vertices + edges; directed/undirected, weighted/unweighted, dense/sparse.
- Adjacency list is the default (O(V+E) space); matrix for dense graphs and O(1) edge checks.
- BFS (queue) = level order = shortest path unweighted; DFS (stack/recursion) = dive deep.
- Dijkstra settles closest-first via min-heap — needs non-negative weights.
- Topological sort linearizes DAGs; leftover nodes mean a cycle.

**Keep exploring**

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev